# INGENIERÍA DE SOFTWARE — APUNTE MAESTRO TEÓRICO

Compendio exhaustivo de lecturas obligatorias, diseño de patrones GoF, Smalltalk-80 y banco de práctica de examen.

## 1. MODERN SOFTWARE ENGINEERING (DAVID FARLEY) — CAP. 1, 2 Y 3

### 1.1. La naturaleza del desarrollo: El software como proceso de aprendizaje continuo

Farley establece que el desarrollo de software no es un problema de producción ni una línea de ensamblaje industrial repetitiva, sino una disciplina fundamentalmente orientada al **diseño y la exploración continua**. Por su naturaleza intangible y flexible, los ingenieros de software se enfrentan constantemente a la incertidumbre y deben convertirse en **expertos en aprender** y en **expertos en gestionar la complejidad**.

**Producción en masa vs. Actividad de diseño:** A diferencia de la ingeniería civil o mecánica tradicional, donde la manufactura física representa el 90% del costo y riesgo, en software la **producción en masa tiene un costo marginal nulo e instantáneo** (copiar bytes, compilar o desplegar). Todo el riesgo, valor y esfuerzo de la disciplina radica exclusivamente en la **actividad de diseño**.

### 1.2. El Método Científico Pragmático y sus Herramientas

Para aprender con rigor y evitar guiarse por conjeturas o dogmas no testeados, la ingeniería de software debe apoyarse en el método científico pragmático aplicado a la resolución de problemas mediante cuatro pasos iterativos: **1) Caracterizar** el problema, **2) Hipotetizar** una solución, **3) Predecir** el resultado y **4) Experimentar** empíricamente. Farley resalta tres pilares epistemológicos:

• **Control de variables:** Diseñar sistemas deterministas y aislar factores causales (ej. automatización de pruebas, ambientes reproducibles y control de dependencias externas).

• **Falsabilidad:** Buscar activamente cómo refutar o poner a prueba nuestras hipótesis de diseño lo más rápido posible, en lugar de intentar confirmar creencias subjetivas.

• **Escepticismo empírico:** Evaluar el software por su comportamiento medible y verificable en ejecución real, descartando argumentos de autoridad.

### 1.3. Crítica a los Métodos Formales y al Modelo en Cascada (Waterfall)

**El fracaso de los Métodos Formales:** Los métodos formales buscan demostrar matemáticamente que un programa es correcto antes de su ejecución. Farley argumenta que en sistemas reales —que son interactivos, concurrentes, distribuidos y sometidos a requerimientos cambiantes— formular una demostración matemática completa es **tan o más complejo y propenso a errores que escribir el propio código**, resultando inviable en términos de costo-beneficio.

**El fallo del Modelo en Cascada:** El modelo secuencial tradicional asume erróneamente que los requerimientos y la arquitectura pueden congelarse de antemano. Al posponer la integración y el feedback real hasta el final del ciclo de vida, la cascada maximiza el riesgo y niega la necesidad empírica de descubrir el diseño iterativamente.

### 1.4. Métricas de Rendimiento de Equipos (Accelerate / DORA)

Farley adopta las métricas de investigación empírica de *Accelerate*, demostrando que **la velocidad y la estabilidad no son fuerzas contrapuestas**, sino que se potencian mutuamente a través del feedback rápido:

| Dimensión | Métrica de Rendimiento | Qué mide y cómo impacta en el diseño |
|---|---|---|
| **Estabilidad (Stability / Calidad)** | • Change Failure Rate<br>• Mean Time to Recovery (MTTR) | Porcentaje de cambios que degradan o fallan en producción, y el tiempo promedio transcurrido entre la detección de un fallo y su resolución efectiva. |
| **Flujo (Throughput / Velocidad)** | • Lead Time for Changes<br>• Deployment Frequency | Tiempo que transcurre desde el commit hasta su ejecución segura en producción, y la cadencia periódica con la que se entrega valor al usuario. |

### 1.5. Las 5 Conductas para Aprender y los 5 Pilares de la Complejidad

**Las 5 conductas para ser expertos en aprender:** 1) Trabajar iterativamente, 2) Obtener feedback rápido y frecuente, 3) Trabajar incrementalmente, 4) Ser experimental, y 5) Ser empírico.

**Los 5 pilares para gobernar la complejidad:** 1) Modularidad, 2) Cohesión, 3) Separación de responsabilidades/incumbencias, 4) Abstracción y ocultamiento de información, y 5) Gestión del acoplamiento.

## 2. ALAN KAY EN QUORA: LA DEFINICIÓN DE 'CIENCIA' EN COMPUTER SCIENCE

**El diagnóstico de Alan Kay:** La educación en ciencias de la computación ha sufrido una **degradación y embrutecimiento desastroso (*dumbing down*)**. La universidad abandonó la aspiración intelectual pionera (de figuras como Licklider, Engelbart o Sutherland) para convertirse en un mero **centro de capacitación técnico-vocacional** subordinado a las modas comerciales inmediatas.

**Causas y Responsables:** 1) Presiones corporativas que demandan programadores rápidos en lenguajes industriales (como C++ o Java) sin fundamentos teóricos profundos, y 2) Universidades complacientes que confundieron la construcción de software instrumental con la ciencia fundamental.

**Definición de Computación según Alan Perlis y Alan Kay:** La computación es **'La ciencia de los procesos; de todos los procesos'**. No es la ciencia de los artefactos físicos (las computadoras), sino el estudio, comprensión, formulación y modelado cualitativo y cuantitativo de **sistemas dinámicos de procesos interactivos que evolucionan en el tiempo**.

**Las Ciencias de lo Artificial (Herb Simon):** A diferencia de la física o biología (que estudian fenómenos preexistentes creados por la naturaleza), la computación crea universos artificiales mediante leyes y relaciones dinámicas inventadas por el hombre. En este contexto, la computadora actúa como un vehículo 'meta': los sistemas no se deducen a priori con matemática estática, sino que se **exploran, formulan y depuran (*debugged*) empíricamente en el espacio de procesos**.

## 3. CULTURES OF PROGRAMMING (TOMAS PETRICEK) — CONCLUSIONES

Petricek analiza 70 años de historia computacional demostrando que la programación es un campo **esencialmente pluralista** moldeado por la interacción de cinco culturas intelectuales estables:

• **Matemática:** El programa es un objeto formal abstracto. Busca la corrección probada a priori mediante lógica deductiva y sistemas de tipos.

• **Hacker:** La programación es una forma de arte y maestría interactiva. Prioriza el control directo del hardware, la exploración libre y la experimentación.

• **Ingenieril:** Concibe el desarrollo como una disciplina pragmática de resolución de problemas bajo restricciones del mundo real, basada en testing, empirismo y herramientas.

• **Gerencial:** Concibe el código como un activo empresarial que requiere predictibilidad, especificaciones contractuales, metodologías formales y control de personal.

• **Humanística:** Ve al programa como un medio para el pensamiento, la expresión creativa y la democratización del aprendizaje (Smalltalk, Dynabook, Alan Kay).

### Los 4 Patrones de Interacción y Choque entre Culturas:

1. **Colaboración alrededor de una entidad compartida:** Múltiples culturas aportan facetas que dan origen a un nuevo concepto frontera (ej. el concepto de *lenguaje de programación* nutriéndose de la portabilidad gerencial, trucos hackers y gramáticas matemáticas).

2. **Cooptación o reinterpretación:** Una cultura toma un concepto nacido en otra y lo redirige a sus propios fines (ej. la *programación estructurada* de Dijkstra cooptada por la cultura gerencial para estructurar jerárquicamente equipos de trabajo).

3. **Lucha por el control de un término:** Disputa política por la hegemonía del significado de un concepto clave (ej. las conferencias NATO de 1968 disputando si 'Ingeniería de Software' era gestión gerencial, verificación formal matemática o pragmatismo técnico).

4. **Desacuerdo conceptual profundo:** Debates originados en asunciones ontológicas incompatibles sobre la naturaleza del software (ej. el debate sobre si la verificación formal es viable frente a la dimensión social y empírica del código).

**Conclusión final: 'Many things go':** Apoyándose en Feyerabend y Chang, Petricek concluye que no existe ni debe imponerse un único dogma supremo; la computación progresa cuando se fomenta el comercio y pluralismo entre culturas.

## 4. CATÁLOGO DE PATRONES DE DISEÑO (GOF) Y MECÁNICAS TEÓRICAS

### Patrón STATE

• **Intención:** Permitir que un objeto altere su comportamiento cuando cambia su estado interno. El objeto parecerá cambiar de clase.

• **Usos conocidos:** Conexiones de red (`TCPConnection` alternando entre `TCPEstablished`, `TCPListen`, `TCPClosed`), parsers de compiladores, flujos de transacciones bancarias.

• **Mecánica teórica:** Aplica el modismo *Handle/Body*. El `Context` delega las peticiones dependientes del estado en una instancia de `ConcreteState`. Reemplaza sentencias condicionales (`switch`/`if`) dispersas por clases cohesivas e independientes. Las transiciones se vuelven explícitas y atómicas.

• **State vs. Tablas de Transición:** Las tablas parametrizan las transiciones mediante datos pero dificultan asociar acciones complejas; **State modela el comportamiento específico de cada estado en objetos**.

### Patrón STRATEGY

• **Intención:** Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo utilizan.

• **Usos conocidos:** Algoritmos de particionado de párrafos (`SimpleCompositor`, `TeXCompositor`), motores de encriptación, validación de formularios, cálculo fiscal.

• **Comparativas clave de examen:**

- *Strategy vs. Template Method:* Strategy usa **composición/delegación** en tiempo de ejecución para variar el algoritmo completo; Template Method usa **herencia estática** para alterar pasos individuales conservando el esqueleto común.

- *Strategy vs. State:* Comparten la misma estructura *Handle/Body*, pero difieren en intención: Strategy es configurado estáticamente por el cliente (*bind-once*); State cambia de forma autónoma a lo largo del tiempo.

- *Strategy vs. Decorator:* 'Strategy cambia las tripas/interior (*guts*) de un objeto; Decorator cambia la piel exterior (*skin*)'.

## Patrón TEMPLATE METHOD

• **Intención:** Definir el esqueleto de un algoritmo en una operación de la clase base, difiriendo ciertos pasos a las subclases sin modificar la estructura del algoritmo.

• **Usos conocidos:** Frameworks de aplicaciones (`Application::OpenDocument` coordinando `DoCreateDocument`, `DoRead`), frameworks de testing (`setUp`, `tearDown`).

• **Mecánica teórica:** Hace cumplir el **Principio de Hollywood** ('No nos llames, nosotros te llamaremos'). Se estructura en: 1) Métodos concretos invariantes, 2) Operaciones primitivas abstractas obligatorias, y 3) Métodos gancho (*hooks*) que ofrecen comportamientos por defecto opcionales.

## Patrón COMPOSITE y el Dilema Safety vs. Transparency

• **Intención:** Componer objetos en estructuras de árbol para representar jerarquías de parte-todo (*part-whole*). Permite a los clientes tratar de manera uniforme a objetos individuales y a composiciones de objetos.

• **Usos conocidos:** Jerarquías de vistas (`View` y `CompositeView` en Smalltalk MVC), sistemas de archivos (directorios y archivos), inventarios de equipamiento, comandos macro.

> **POSTURA DE LA CÁTEDRA SOBRE SAFETY VS. TRANSPARENCY:**
> • **Transparencia (GoF por defecto):** Declara la gestión de hijos (add:, remove:) en la raíz Component. Otorga uniformidad de interfaz, pero es insegura en compilación (las hojas heredan métodos absurdos que deben fallar o ser no-ops).
> • **Seguridad (Postura Elegida por la Cátedra):** Declara la gestión de hijos **únicamente en la clase Composite**.
> **Justificación Teórica Obligatoria:** Se elige **Seguridad** porque respeta el principio ontológico fundamental de que *un objeto solo debe entender y responder los mensajes que semánticamente sabe responder*. Forzar a una Leaf a recibir add: rompe la cohesión semántica y degrada el diseño.

## Patrón OBSERVER

• **Intención:** Definir una dependencia de uno a muchos entre objetos, de modo que cuando un objeto cambie su estado, todos sus dependientes sean notificados y actualizados automáticamente.

• **Usos conocidos:** Arquitectura MVC (Model como Subject, Views como Observers); toolkits de UI (InterViews, ET++); arquitecturas Publish-Subscribe reactivas.

• **Modelo Push vs. Pull:** En *Push*, el sujeto envía los datos detallados en el mensaje de notificación (acopla al observador); en *Pull*, el sujeto sólo avisa del cambio y el observador consulta activamente lo que necesita mediante getters (máximo desacoplamiento).

• **ChangeManager (Mediator):** Centraliza dependencias complejas muchos-a-muchos para evitar notificaciones redundantes y coordinar actualizaciones cíclicas.

### Ejemplo Declarativo del Patrón Observer (Smalltalk):

```
"Sujeto de Dominio"
SensorTemperatura >> registrarObservador: unObservador
    observadores add: unObservador.

SensorTemperatura >> temperatura: unValor
    temperatura := unValor.
    self notificar.

SensorTemperatura >> notificar
    observadores do: [ :cadaUno | cadaUno actualizarCon: self ].

"Observador Concreto"
TermostatoVisual >> actualizarCon: unSensor
    self mostrarEnPantalla: unSensor temperatura.
```

## Patrón DECORATOR

• **Intención:** Adjuntar responsabilidades adicionales a un objeto de forma dinámica y transparente. Proporciona una alternativa flexible a la herencia para extender funcionalidad.

• **Usos conocidos:** Bordes y scrolls en componentes gráficos (`BorderDecorator`, `ScrollDecorator` sobre `TextView`); streams de I/O (`CompressingStream` sobre `FileStream`).

• **Mecánica teórica:** Conserva exactamente la **misma interfaz** del componente que envuelve, permitiendo anidamiento y composición recursiva ilimitada.

### Ejemplo Declarativo del Patrón Decorator (Smalltalk):

```
"Componente Concreto"
MensajeSimple >> texto
    ^ 'hola mundo'

"Decorador Base y Decoradores Concretos"
DecoradorMensaje >> initializeCon: unComponente
    envoltorio := unComponente.

MensajeMayusculas >> texto
    ^ envoltorio texto asUppercase.

MensajeConFirma >> texto
    ^ envoltorio texto , ' - Atte: Juan'

"Composición Dinámica y Recursiva"
mensajeFinal := MensajeConFirma sobre: (MensajeMayusculas sobre: MensajeSimple new).
mensajeFinal texto.  "Retorna: 'HOLA MUNDO - Atte: Juan'"
```

## Patrones ADAPTER y PROXY (Comparativa de Wrappers)

• **Adapter:** Convierte la interfaz de una clase en otra interfaz que el cliente espera para resolver incompatibilidades de diseño entre clases preexistentes.

• **Proxy:** Proporciona un representante de otro objeto para controlar o diferir el acceso a él sin alterar su interfaz (Virtual Proxy, Remote Proxy, Protection Proxy).

| Patrón | Relación con la Interfaz | Intención Primaria | Composición Recursiva |
|---|---|---|---|
| **Adapter** | **Diferente** (Adapta al target) | Resolver incompatibilidad entre clases ya creadas | No admitida |
| **Decorator** | **Misma o enriquecida** | Agregar responsabilidades dinámicamente | Sí (ilimitada y transparente) |
| **Proxy** | **Exactamente la misma** | Controlar o diferir el acceso al objeto real | Rara vez / No es su propósito |

### Proxy Polimórfico en Smalltalk-80 (superclass: nil):

Un proxy que hereda de `Object` no es totalmente transparente: intercepta solo mensajes específicos, mientras que mensajes nativos (`isNil`, `class`, `printString`) son respondidos por el proxy y nunca llegan al objeto real. Para resolver esto, se define el proxy con **`superclass: nil`** (o sobre `ProtoObject`). Al carecer de tabla de métodos heredada, **cualquier mensaje** falla el lookup y es capturado por **`doesNotUnderstand: aMessage`**, permitiendo reenviarlo de forma completamente genérica mediante **`realSubject perform: aMessage selector withArguments: aMessage arguments`**.

## Patrón VISITOR

• **Intención:** Representar una operación a realizar sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.

• **Usos conocidos:** Compiladores y AST (`TypeCheckingVisitor`, `CodeGeneratingVisitor`), cálculo impositivo sobre jerarquías de productos.

• **Mecanismos subyacentes de examen:**

1. **Double Dispatch (Doble Despacho):** La operación concreta depende del tipo en tiempo de ejecución de dos receptores: el `Visitor` y el `Element` (`element accept: aVisitor` \(\rightarrow\) `aVisitor visitConcreteElement: self`).
2. **Composite:** La estructura que suele atravesar el Visitor es casi siempre una jerarquía arbórea modelada con Composite.

• **Matriz de Trade-offs:** Es muy fácil agregar **nuevas operaciones** (basta con crear una subclase de Visitor), pero es muy costoso agregar **nuevos elementos concretos** (obliga a modificar la interfaz Visitor y recompilar todas sus subclases). Además, rompe el encapsulamiento de los elementos.

## Patrón BUILDER

• **Intención:** Separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda crear diferentes representaciones.

• **Usos conocidos:** Conversores de texto (un `RTFReader` comanda `TeXConverter` o `ASCIIConverter`), parsers de estructuras complejas.

• **Mecánica teórica:** El `Director` ejecuta la secuencia invariante de ensamblado paso a paso, mientras que el `ConcreteBuilder` acumula el estado y entrega el producto terminado. A diferencia de Abstract Factory (que entrega productos inmediatos independientes), Builder construye un único objeto complejo por etapas.

# 5. ACTIVE VARIABLES, FAIL FAST Y ANTI-PATRONES DE DISEÑO

## 5.1. Active Variables in Smalltalk-80 (Messick & Beck)

Mecanismo a nivel de **lenguaje y compilador** para reificar la lectura y asignación de variables de memoria ejecutando procedimientos auxiliares llamados **daemons** sin modificar el código fuente de los objetos ni contaminar su dominio con llamadas manuales a observadores.

• **Get Daemon:** Procedimiento que se invoca automáticamente al evaluar/leer la variable.

• **Set Daemon:** Procedimiento que se invoca automáticamente al asignar un nuevo valor a la variable.

• **Jerarquía de Metamodelo:** `Object` \(\rightarrow\) `InactiveVariable` (desactiva monitoreos sin recompilar) \(\rightarrow\) `ActiveVariable` (daemons como selectores de método) \(\rightarrow\) `SpecificActiveVariable` (daemons como bloques de Smalltalk que capturan el contexto).

• **¿Por qué NO usa ProtoObject?** La intercepción la realiza el compilador reescribiendo los accesos de bytecode, no mediante fallos en `doesNotUnderstand:`. Además, requiere variables de instancia propias (`name`, `getDaemon`, `setDaemon`).

• **Casos de uso:** Paquete de *Dials and Gauges* (medidores gráficos reactivos), validación de tipos en runtime, breakpoints condicionales y *Advised Methods* (inyección de código antes/después de un método sin recompilarlo).

## 5.2. Fail Fast y Falla Robusta (Jim Shore)

Un sistema debe fallar de manera **inmediata y visible** apenas se detecta una condición inconsistente, utilizando aserciones con mensajes contextuales claros. Se opone radicalmente al *Fail Slowly* (ocultar errores con códigos de retorno o try/catch vacíos que corrompen el estado y fallan lejos del bug).

**Falla Robusta en Producción (*Robust Failure*):** Para evitar caídas totales de la aplicación, el autor propone implementar un **Manejador Global de Errores (*Global Error Handler*)** en la capa superior de eventos. Este captura la excepción no controlada, registra la información detallada para los desarrolladores, aísla la transacción fallida y mantiene al sistema en pie para procesar las siguientes peticiones. Exige erradicar los `catch-all` locales y liberar recursos con bloques `finally` o sentencias `using`.

## 5.3. Pattern Abuser: La Falacia de la Métrica de Patrones

Satiriza la idea ingenua de que *'a mayor cantidad de patrones implementados, mayor es la calidad del software'*. La sobre-ingeniería decorativa genera capas excesivas de indirección innecesaria, provoca explosión de clases, dificulta la legibilidad del dominio y arruina la mantenibilidad. Los patrones sólo se justifican ante problemas reales de acoplamiento y evolución.

## 6. BANCO DE EXAMEN TEÓRICO: PREGUNTAS DE PRÁCTICA DE PARCIAL

Cuestionario elaborado siguiendo rigurosamente el formato y estilo de evaluación teórica de la cátedra en la UBA. Utilízalo para ejercitar y evaluar tu comprensión antes del examen:

**Pregunta 1 (Farley - Producción vs. Diseño):**
Explique por qué David Farley sostiene que la ingeniería de software se diferencia radicalmente de las ingenierías tradicionales en lo relativo a la 'producción en masa'. ¿Dónde radica el riesgo y el costo en nuestra disciplina?

**Pregunta 2 (Farley - Métodos Formales vs. Cascada):**
¿Por qué los métodos formales matemáticos no escalan para garantizar la corrección de sistemas de software complejos del mundo real? ¿Qué falacia fundamental comparte el modelo en Cascada (*Waterfall*) con respecto a la naturaleza del desarrollo?

**Pregunta 3 (Farley - Métricas DORA):**
Defina las dos dimensiones propuestas para medir el desempeño de equipos de software (*Stability* y *Throughput*). Mencione dos métricas cuantitativas que componen cada dimensión y explique por qué velocidad y calidad no son trade-offs contrapuestos.

**Pregunta 4 (Alan Kay - Computer Science):**
Según Alan Kay en su respuesta de Quora, ¿por qué la educación universitaria actual ha sufrido un 'embrutecimiento' (*dumbing down*)? Citando a Alan Perlis, ¿cómo debe definirse la computación y en qué se diferencia de una ciencia natural tradicional?

**Pregunta 5 (Petricek - Culturas de Programación):**
Describa brevemente cómo conciben la noción de 'programa' la cultura Matemática, la cultura Hacker y la cultura Gerencial. ¿Qué significa la conclusión 'Many things go' formulada por el autor?

**Pregunta 6 (Petricek - Patrones de Choque):**
Explique la diferencia entre el patrón de 'Cooptación o Reinterpretación' y el de 'Lucha por el Control de un Concepto'. Ilustre cada uno con un ejemplo histórico analizado en el texto.

**Pregunta 7 (State vs. Strategy vs. Template Method):**
Compare detalladamente los patrones State, Strategy y Template Method en términos de: 1) Mecanismo de variabilidad empleado (herencia vs. composición), 2) Dinamismo temporal (en qué momento y con qué frecuencia cambia el comportamiento), y 3) Quién gobierna el cambio.

**Pregunta 8 (Composite - Safety vs. Transparency):**
Explique el dilema de diseño 'Safety' vs. 'Transparency' en el patrón Composite. ¿Cuál es la postura adoptada por la cátedra y cuál es la justificación teórica y ontológica para fundamentar dicha elección?

**Pregunta 9 (Observer vs. Active Variables):**
Compare el patrón Observer (GoF) con el mecanismo de Active Variables en Smalltalk-80 considerando: nivel en el que operan (aplicación vs. compilador/entorno), acoplamiento entre emisor y receptor, y el rol de las daemons.

**Pregunta 10 (Wrappers - Proxy, Decorator y Adapter):**
Construya un cuadro comparativo entre los patrones Adapter, Decorator y Proxy analizando: 1) Cómo se comporta su interfaz respecto a la del objeto envuelto, 2) Cuál es su intención de diseño principal, y 3) Si admiten o no composición recursiva transparente.

**Pregunta 11 (Proxy Polimórfico en Smalltalk):**
¿Por qué un Proxy que hereda de la clase `Object` en Smalltalk no es completamente polimórfico? ¿Qué mecanismo arquitectónico se utiliza para solucionar esta limitación y qué rol cumple el mensaje `doesNotUnderstand:`?

**Pregunta 12 (Visitor - Mecanismos y Trade-offs):**
¿Qué dos mecanismos o patrones de diseño subyacen en una solución modelada con Visitor? Explique la matriz de trade-offs de extensibilidad: ¿qué tipo de cambio resulta sumamente económico y qué tipo de cambio resulta sumamente costoso?

**Pregunta 13 (Active Variables - Metamodelo):**
Explique la jerarquía de clases de Active Variables (`InactiveVariable`, `ActiveVariable`, `SpecificActiveVariable`). ¿Por qué una Active Variable no debe ni necesita heredar de `ProtoObject`?

**Pregunta 14 (Fail Fast vs. Falla Robusta):**
¿En qué consiste el principio Fail Fast propuesto por Jim Shore y por qué se opone al 'Fail Slowly'? ¿Cómo se logra una 'Falla Robusta' en producción mediante un Manejador Global de Errores sin que el sistema colapse por completo?

**Pregunta 15 (Pattern Abuser):**
¿Qué falacia de medición de calidad critica el artículo 'Pattern Abuser' y qué consecuencias negativas introduce el uso indiscriminado y preventivo de patrones de diseño en un sistema de software?